Top 50 SQL Interview Questions with Answers

1. Find the Second Highest Salary

Answer

```
SELECT MAX(salary)
FROM Employee
WHERE salary <
(
    SELECT MAX(salary)
    FROM Employee
);
```

2. Find Nth Highest Salary

Answer

```
SELECT salary
FROM
(
    SELECT salary,
           DENSE_RANK() OVER(ORDER BY salary DESC) rnk
    FROM Employee
) t
WHERE rnk = N;
```


3. Highest Salary in Each Department

Answer

```
SELECT department,
       MAX(salary)
FROM Employee
GROUP BY department;
```

4. Second Highest Salary in Each Department

Answer

```
SELECT *
FROM
(
    SELECT *,
           DENSE_RANK() OVER
           (
              PARTITION BY department
              ORDER BY salary DESC
           ) rnk
    FROM Employee
)t
WHERE rnk=2;
```

5. Top 3 Highest Salaries in Each Department

Answer

```
SELECT *
FROM
(
SELECT *,
       DENSE_RANK() OVER
       (
           PARTITION BY department
           ORDER BY salary DESC
       ) rnk
FROM Employee
)t
WHERE rnk<=3;
```


6. Employees Earning More Than Department Average

Answer

```
SELECT *
FROM Employee e
WHERE salary >
(
SELECT AVG(salary)
FROM Employee
WHERE department=e.department
);
```

7. Employees Earning Less Than Department Average

Answer

```
SELECT *
FROM Employee e
WHERE salary <
(
SELECT AVG(salary)
FROM Employee
WHERE department=e.department
);
```

8. Employee Having Highest Salary

Answer

```
SELECT *
FROM Employee
ORDER BY salary DESC
LIMIT 1;
```


9. Employee Having Lowest Salary

Answer

```
SELECT *
FROM Employee
ORDER BY salary
LIMIT 1;
```

10. Find Duplicate Records

Answer

```
SELECT email,
COUNT(*)
FROM Employee
GROUP BY email
HAVING COUNT(*)>1;
```

11. Delete Duplicate Records

Answer

```
DELETE FROM Employee
WHERE id NOT IN
(
SELECT MIN(id)
FROM Employee
GROUP BY email
);
```

12. Find Employees Without Manager

Answer

```
SELECT *
FROM Employee
WHERE managerid IS NULL;
```


13. Employees Earning More Than Their Manager

Answer

```
SELECT e.*
FROM Employee e
JOIN Employee m
ON e.managerid=m.employeeid
WHERE e.salary>m.salary;
```

14. Employees Joined in Last 30 Days

Answer

```
SELECT *
FROM Employee
WHERE joiningdate>=CURRENT_DATE-INTERVAL '30 day';
```

15. Count Employees in Each Department

Answer

```
SELECT department,
COUNT(*)
FROM Employee
GROUP BY department;
```

16. Find Departments Having More Than 10 Employees

Answer

```
SELECT department,
COUNT(*)
FROM Employee
GROUP BY department
HAVING COUNT(*)>10;
```


17. Find Duplicate PAN Numbers

Answer

```
SELECT pannumber,
COUNT(*)
FROM Customer
GROUP BY pannumber
HAVING COUNT(*)>1;
```

18. Find Missing IDs

Answer

```
SELECT id+1
FROM Employee
WHERE id+1 NOT IN
(
SELECT id
FROM Employee
);
```

19. ROW_NUMBER()

Assigns unique numbers.

Answer

```
SELECT *,
ROW_NUMBER() OVER(ORDER BY salary DESC)
FROM Employee;
```

20. RANK()

Answer

```
SELECT *,
RANK() OVER(ORDER BY salary DESC)
FROM Employee;
```


21. DENSE_RANK()

Answer

```
SELECT *,
DENSE_RANK() OVER(ORDER BY salary DESC)
FROM Employee;
```

22. LAG()

Answer

```
SELECT employeeid,
salary,
LAG(salary) OVER(ORDER BY salary)
FROM Employee;
```

23. LEAD()

Answer

```
SELECT employeeid,
salary,
LEAD(salary) OVER(ORDER BY salary)
FROM Employee;
```

24. FIRST_VALUE()

Answer

```
SELECT *,
FIRST_VALUE(salary)
OVER(ORDER BY salary DESC)
FROM Employee;
```


25. LAST_VALUE()

Answer

```
SELECT *,
LAST_VALUE(salary)
OVER(
ORDER BY salary
ROWS BETWEEN UNBOUNDED PRECEDING
AND UNBOUNDED FOLLOWING
)
FROM Employee;
```

26. Running Total

Answer

```
SELECT employeeid,
salary,
SUM(salary)
OVER(ORDER BY employeeid)
FROM Employee;
```

27. Moving Average

Answer

```
SELECT salary,
AVG(salary)
OVER(
ORDER BY employeeid
ROWS BETWEEN 2 PRECEDING
AND CURRENT ROW
)
FROM Employee;
```

28. Find Customers Without Orders

Answer

```
SELECT *
FROM Customer c
LEFT JOIN Orders o
ON c.customerid=o.customerid
WHERE o.customerid IS NULL;
```

29. Find Orders Without Customer

Answer

```
SELECT *
FROM Orders o
LEFT JOIN Customer c
ON o.customerid=c.customerid
WHERE c.customerid IS NULL;
```

30. Inner Join Example

Answer

```
SELECT *
FROM Customer
INNER JOIN Orders
USING(customerid);
```

31. Left Join

Answer

```
SELECT *
FROM Customer
LEFT JOIN Orders
USING(customerid);
```

32. Right Join

Answer

```
SELECT *
FROM Customer
RIGHT JOIN Orders
USING(customerid);
```

33. Full Join

Answer

```
SELECT *
FROM Customer
FULL JOIN Orders
USING(customerid);
```

34. Cross Join

Answer

```
SELECT *
FROM Employee
CROSS JOIN Department;
```

35. Self Join

Answer

```
SELECT e.employeeid,
m.employeeid manager
FROM Employee e
LEFT JOIN Employee m
ON e.managerid=m.employeeid;
```

36. EXISTS Example

Answer

```
SELECT *
FROM Customer c
WHERE EXISTS
(
SELECT 1
FROM Orders o
WHERE o.customerid=c.customerid
);
```


37. NOT EXISTS Example

Answer

```sql
SELECT *
FROM Customer c
WHERE NOT EXISTS
(
SELECT 1
FROM Orders o
WHERE o.customerid=c.customerid
);
```

38. UNION vs UNION ALL

Answer

```sql
SELECT city FROM Customer
UNION
SELECT city FROM Supplier;
```

39. Common Table Expression (CTE)

Answer

```sql
WITH cte AS
(
SELECT *,
ROW_NUMBER() OVER(ORDER BY salary DESC) rn
FROM Employee
)
SELECT *
FROM cte;
```


40. Recursive CTE

Answer

```
WITH RECURSIVE nums AS
(
SELECT 1 n
UNION ALL
SELECT n+1
FROM nums
WHERE n<10
)
SELECT *
FROM nums;
```

41. Pivot Data

Use conditional aggregation.

Answer

```
SELECT
SUM(CASE WHEN gender='M' THEN 1 END) Male,
SUM(CASE WHEN gender='F' THEN 1 END) Female
FROM Employee;
```

42. Find Even Records

Answer

```
SELECT *
FROM Employee
WHERE MOD(employeeid,2)=0;
```

43. Find Odd Records

Answer

```
SELECT *
FROM Employee
WHERE MOD(employeeid,2)=1;
```

44. Latest Record per Customer

Answer

```
SELECT *
FROM
(
SELECT *,
ROW_NUMBER() OVER
(
PARTITION BY customerid
ORDER BY createddate DESC
) rn
FROM Orders
)t
WHERE rn=1;
```

45. Find Consecutive Records

Answer

```
SELECT *
FROM
(
SELECT *,
id-ROW_NUMBER() OVER(ORDER BY id) grp
FROM Employee
)t;
```


46. Difference Between RANK(), DENSE_RANK(), ROW_NUMBER()

| Function | Duplicate Rank | Skip Rank | | - | -- | | | ROW_NUMBER | No | No | | RANK | Yes | Yes | | DENSE_RANK | Yes | No |

47. DELETE vs TRUNCATE vs DROP

DELETE	TRUNCATE	DROP
Removes selected rows	Removes all rows	Removes table
Can use WHERE	No WHERE	Deletes structure
Rollback possible (transaction dependent)	Often minimally logged	Removes object

48. Clustered vs Non-Clustered Index

Clustered

- Data stored in index order
- One per table

Non-Clustered

- Separate index structure
- Multiple allowed

49. Primary Key vs Unique Key

| Primary | Unique | | -- | | | No NULL | NULL allowed (DB-dependent) | | One per table | Multiple | | Uniquely identifies row | Enforces uniqueness |

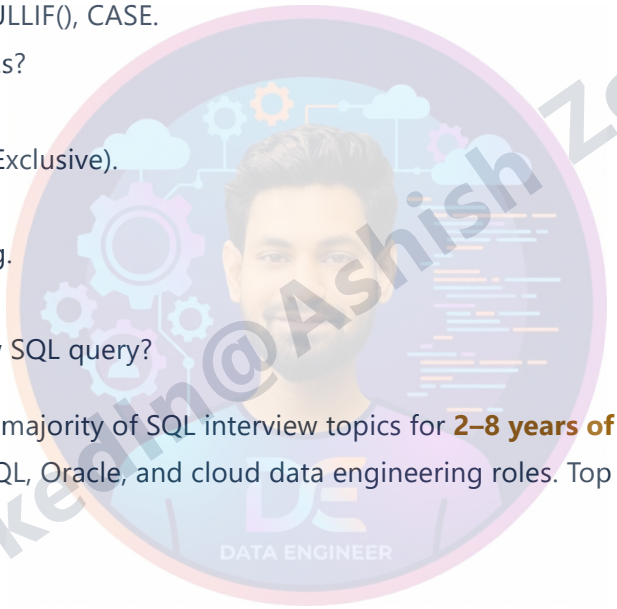
50. Explain SQL Execution Order

Logical execution order:

1. FROM
2. JOIN
3. WHERE
4. GROUP BY
5. HAVING
6. SELECT
7. DISTINCT
8. ORDER BY
9. LIMIT / OFFSET

Bonus Advanced Questions

- What are Window Functions?
- Explain ACID Properties.
- What is Normalization?
- Explain Denormalization.
- What is Indexing?
- Explain Composite Index.
- What is Covering Index?
- What is Query Optimization?
- What is Execution Plan?
- What are Materialized Views?
- Difference between CHAR and VARCHAR.
- What are Correlated Subqueries?
- Explain COALESCE(), NULLIF(), CASE.
- What are Recursive CTEs?
- What are Transactions?
- Explain Locks (Shared, Exclusive).
- Deadlock vs Blocking.
- Partitioning vs Sharding.
- OLTP vs OLAP.
- How to optimize a slow SQL query?



These 50 questions cover the majority of SQL interview topics for **2–8 years of experience**, especially for PostgreSQL, SQL Server, MySQL, Oracle, and cloud data engineering roles. Top